

Neq Array - omówienie

INOI 2025
18 stycznia 2026

Kod zadania: **neq**
Limit pamięci: **1024 MiB**



Treść

Mamy dany ciąg A o długości n i q zapytań o jego przedziały ($1 \leq n, q \leq 200\,000, 1 \leq A_i \leq n$).

Weźmy ciąg liczb całkowitych C o długości m . Ciąg B nazwiemy *dobrym* dla C , jeżeli:

- ciągi B i C mają tę samą długość
- zachodzi $0 < B_1 \leq B_2 \leq \dots \leq B_m$
- $B_i \neq C_i$ dla każdego $1 \leq i \leq m$

Niech $f(C)$ oznacza najmniejszą wartość B_m po wszystkich ciągach B *dobrych* dla C . Chcemy odpowiadać na zapytania o wartości $f(A[l, r])$.

Strategia zachłanna

Zastanówmy się jak konstruować optymalne *dobre* ciągi. Spróbujemy to robić w sposób zachłanny – ustawimy B_i na B_{i-1} jeżeli jest różne od A_i , w przeciwnym przypadku na $A_i + 1$.

Przykład: ciąg $[2, 1, 1, 3, 2, 1]$ z testu przykładowego wygeneruje nam *dobry* ciąg $[1, 2, 2, 2, 3, 3]$.

Nietrudno zauważyć, że w ten sposób otrzymamy poprawną odpowiedź. Dowód pozostawiam jako ćwiczenie dla czytelnika.

Pozostaje tylko problem złożoności: na razie na każde zapytanie musimy odpowiadać w czasie liniowym.

Wzorcówka

Znając już optymalną strategię, możemy nieco inaczej przedstawić nasz oryginalny problem: na pewnym przedziale ciągu A szukamy długości jego najdłuższego podciągu o postaci $1, 2, \dots$.

Co by było, gdybyśmy trzymali dla każdego indeksu i wskaźnik do najbliższego na prawo indeksu j takiego, że $A_i + 1 = A_j$? Wystarczyłoby wtedy znaleźć pierwszą jedynekę na przedziale i skakać po wskaźnikach aż nie przekroczymy końca przedziału.

Spróbujmy jeszcze szybciej – zamiast wykonywać wiele skoków, najpierw zbudujemy sobie na nich jump-pointery, a odpowiadając na zapytania możemy zrobić binary-lifting. Teraz obie części działają w logarytmicznej złożoności (pierwszą jedynekę szukamy jakimś lower boundem).

Ostateczna złożoność: $O((n + q) \cdot \log_2 n)$

